FaultPilot: Evidence-Grounded Debugging for Python AI Applications

Phase 1 — Literature Review, Research Gaps and Project Proposal

Fazal Zaman Khan (22I-2362)

9 October 2026

Course: AI4015 — Agentic Artificial Intelligence
Project code: G2-08
Project seed: Python AI/ML fault diagnosis and repair
Group: Solo — Fazal Zaman Khan (22I-2362)
Supervisor: Meemon Khan (G2 projects, as listed in the course brief)
Repository: https://github.com/FazalZamanKhan/faultpilot-g2-08

Contents

A. Literature Review and Research Gaps **2**
A.1 Scope and selection . . . 2
A.2 Critical paper reviews . . . 2
1. RepairAgent — base paper [1] . . . 2
2. UniDebugger [2] . . . 2
3. COAST and DEBUGEVAL [3] . . . 3
4. SynFix [4] . . . 3
5. MLDebugging [5] . . . 3
6. Hierarchical Knowledge Injection [6] . . . 4
7. Agent trajectory study [7] . . . 4
8. GraphGlue [8] . . . 4
9. RepoDebug [9] . . . 5
10. DSDBench [10] . . . 5
A.3 Cross-paper comparison . . . 5
A.4 Three selected research gaps . . . 7

B. Project Proposal **8**
B.1 Project, problem and target user . . . 8
B.2 Base paper, extension and agentic justification . . . 8
B.3 Confirmed data and Phase 1 baseline . . . 8
B.4 System architecture and data flow . . . 9
B.5 Research questions, hypotheses and success criteria . . . 11
B.6 Evaluation protocol . . . 12
B.7 Milestones, resources and feasibility . . . 12
B.8 Potential publication direction . . . 13

References **14**

A. Literature Review and Research Gaps

A.1 Scope and selection

The review focuses on debugging and repair systems that can inform a Python AI application assistant: agent control, fault localization, contextual evidence, executable verification, and benchmarks with multi-library or data science failures. The ten selected papers appeared in peer-reviewed ICSE, ASE, EMNLP, or ACL-family proceedings in 2025. A paper's published performance is reported in its own setting; numbers obtained on Java Defects4J, repository issues, and Python data science code are not directly comparable. *RepairAgent* [1] is the base paper because its tool-using repair loop is a concrete design to adapt and challenge. ICSE 2025 lists it in the research track, and the official CORE 2023 ranking lists ICSE as A* [11], [12].

A.2 Critical paper reviews

1. RepairAgent — base paper [1]

Bouzenia, Devanbu, and Pradel ask whether an LLM can control a repair process instead of receiving a fixed sequence of prompts. RepairAgent gives the model tools for reading code, searching for repair ingredients, writing patches, and running tests; middleware and a state machine constrain how those tools are used. On all 835 Defects4J Java bugs, the paper reports 186 test-plausible patches and 164 manually judged correct fixes. Its ablation on 100 bugs falls from 21 correct fixes to 11 without search tools and 14 without the state machine. The study also reports 13 correct fixes among 100 GitBug-Java cases and roughly 270,000 model tokens per bug. These figures make a useful case for adaptive evidence gathering, while exposing a real resource cost.

The authors note possible training-data leakage, reliance on pre-existing failing tests, imperfect fault localization, and nondeterministic outputs. I would add that Java projects with curated test harnesses are a poor direct proxy for exploratory Python AI scripts, where package state, tensor shape, data values, and notebook execution order often matter. FaultPilot therefore uses the paper as an architectural base, not as a numerical score to beat without a shared dataset. Its first comparison will be a simpler taxonomy-only pipeline; a later comparison can implement a bounded RepairAgent-style tool loop on the same Python cases and budget.

2. UniDebugger [2]

UniDebugger treats debugging as a coordinated sequence of roles rather than asking several agents to debate as peers. Its hierarchy connects localization, explanation, and patch generation with external tools. The evaluation covers Defects4J, Codeflaws, QuixBugs, and ConDefects. On Defects4J, it reports 197 correct and 286 plausible repairs, 25.48% more correct fixes than the strongest comparator named in its analysis. On Codeflaws, 3,130 of 3,982 programs received plausible fixes; the reported correctness check sampled 100 of those patches and judged 95 correct. Those details matter: a plausible patch count and a sampled correctness rate answer different questions.

The authors explicitly limit their design to test-driven bugs. They do not establish that a generated patch is safe when the supplied suite misses behavior, and their strongest repository result is still in Java. Their multiple-agent arrangement also imposes more orchestration than a narrow Python error may need. For FaultPilot, the lesson is to separate localization and repair responsibilities while allowing a short path for obvious faults. The paper supports a staged workflow, but does not by itself prove that adding agents improves a Python AI diagnosis; that question must be tested against a fixed, less complex baseline.

3. COAST and DEBUGEVAL [3]

Yang and colleagues argue that code repair scores conceal weaknesses in earlier debugging stages. They introduce DEBUGEVAL with 5,712 test examples across Python, C++, and Java and evaluate localization, bug identification, repair, and recognition separately. COAST uses interacting quizzer, learner, and teacher roles to synthesize supervision for small models. In their Table 4, the DeepSeek-Coder 6.7B variant rises from a 28.7 average task score without fine-tuning to 56.4 as NeuDebugger; its repair subscore rises from 40.6 to 49.5. The table also shows that different training strategies help different stages, so an overall score should not hide a weak diagnosis step.

The authors state that the benchmark covers only four tasks and that synthetic-data quality depends on the quizzer and teacher models. Many items are contest-style snippets, which leaves uncertainty about environment failures in installed Python libraries. FaultPilot will borrow the idea of stage-level measurement: category diagnosis, evidence selection, repair attempt, and verification should be evaluated separately. The project will not train a model using COAST in Phase 1; its relevance is the evaluation design and the warning that fluent repair text may coexist with poor fault identification.

4. SynFix [4]

SynFix addresses repairs that span related files and symbols. It constructs a RelationGraph of classes, functions, variables, and dependencies, uses that structure to localize an issue and propagate related edits, then validates patches with regression tests. On SWE-bench Lite it resolves 157 of 300 issues (52.33%) with a reported average model cost of USD 0.56 per issue. The paper also reports 55.8% on SWE-bench Verified and 29.86% on the full set. These are issue-resolution rates under that benchmark's test harness, not a measure of root-cause explanation quality. Its comparison with tool-heavy agents is particularly relevant to a solo project with limited time and API budget: structured retrieval may avoid unnecessary action loops.

The authors identify dependence on regression tests and on an accurate representation of code dependencies. Domain-specific repositories and weak documentation can undermine either assumption. The study does not test whether a proposed fix remains useful when a Python library version changes or when no reliable tests exist. FaultPilot will compare its planned retrieval and verification stages with a no-retrieval, one-shot path. SynFix also motivates retaining evidence links in the report so a user can inspect why the system selected a code region before accepting a repair.

5. MLDebugging [5]

MLDebugging is the closest published data source to FaultPilot's target setting. Huang and colleagues build a Python benchmark for programs using multiple libraries, with 1,175 released examples, 126 libraries, and seven fault categories. They supply buggy and reference code, tests, runtime feedback, and category labels. Their Table 5 reports a 58.7% average test pass rate for DeepSeek-V3, while the 7B Qwen2.5 model reaches 42.7%; requirement misunderstanding and import cases remain difficult. The result shows that even high-capacity models do not simply solve multi-library debugging by reading a traceback.

The authors acknowledge that much of the benchmark is model-generated and that reproducing external environments is costly. Inspecting the pinned public JSONL also revealed practical label hygiene issues: 31 records contain more than one recognizable category mention, and two have no runtime feedback. These counts are a repository data-profile observation, not a claim in the paper. They affect how a taxonomy classifier should be scored. FaultPilot will use this dataset as its primary confirmed source, preserve multi-label cases, and split by source task identifier so related variants cannot leak between development and test sets. The current four-case baseline has not yet been evaluated on this benchmark.

6. Hierarchical Knowledge Injection [6]

Ehsani and colleagues study which context helps an LLM repair Python bugs. Their method layers local bug facts, repository relations and commit history, then project documentation and earlier fixes. On 314 BugsInPy cases, the Llama 3.3 fix count moves from 207 with bug-level knowledge to 235 after repository knowledge and 250 after project knowledge. The final 250/314 corresponds to 79% fixed by the paper's test criterion. Giving all layers at once instead reaches 207/314 in their Table II, a useful reminder that more retrieved text can distract rather than help. The authors also analyze six bug types and find that deeper context helps some types more than others.

Their threats include manually assigned bug labels and the fact that passing tests approximates, rather than proves, semantic correctness. Some difficult program-anomaly and GUI bugs remain unresolved. Because the tested model and its context window differ from RepairAgent's, this paper should not be treated as a direct leaderboard comparison. FaultPilot will use it to justify conditional retrieval: first gather traceback and local code, then seek library or repository evidence if the diagnosis remains uncertain. A measured ablation will test whether that second step improves category accuracy or merely raises latency and token use.

7. Agent trajectory study [7]

Bouzenia and Pradel inspect what software engineering agents actually do during repair. They normalize logs from RepairAgent, AutoCodeRover, and OpenHands into 120 thought-action-result trajectories containing 2,822 LLM interactions. Their reported means differ sharply: RepairAgent takes 34 iterations, OpenHands 29, and AutoCodeRover 6; mean token use is about 220,000, 1.2 million, and 23,000 respectively. The study links repeated actions, misread feedback, and thought-action mismatches with wasted effort, while noting that action sequences vary with each tool design. These observations are system measurements, not a new patch-success benchmark.

The authors caution that only three agents were sampled, semantic coding can be subjective, and correlations do not establish why an attempt succeeds. Their agents also use different backbones and tasks, so their raw costs are not controlled comparisons. For FaultPilot, the practical implication is a bounded state machine and an audit trail: every tool call, result, and next decision should be recorded. A later evaluation can count repeated calls and wasted attempts per case. This paper gives a concrete reason to measure tool efficiency and to stop a loop when the evidence has stopped changing.

8. GraphGlue [8]

Wang and colleagues target a distinctly AI-specific failure: Python constructs that break a deep-learning compiler's computation graph. GraphGlue uses multiple agents, graph-break cause mining, and self-correction with reject sampling to alternate between diagnosis and code regeneration. The ASE 2025 research-paper record reports optimization of 92.63% of 1,411 real user programs. Relative to TorchDynamo directly, the abstract reports up to 2.19 times speedup and 1.23 times on average; it also reports an average 8.74 times memory saving against the chosen compiler-front-end comparison. These are performance and optimization figures, not general bug repair accuracy.

GraphGlue is valuable because it connects an error cause to a concrete execution check in a deep-learning setting. Its scope is also narrow: graph capture and compiler optimization are different from a missing package, device mismatch, or wrong data transformation. The reported aggregate does not establish that its agents handle those other failure classes. FaultPilot will treat graph breaks as an optional extension rather than claim that a general taxonomy already solves them. The method suggests one useful design principle for the core project: a candidate diagnosis should be tied to an executable check that can reject an attractive but ineffective change.

9. RepoDebug [9]

RepoDebug tests whether language models can debug beyond a single function. Liu and colleagues create cases from 63 public repositories, with 22 injected error subtypes across eight languages and tasks for identification, localization, and repair. They evaluate ten models. In the Python rows of Table 3, Claude 3.5 Sonnet reaches 36.77% bug-identification accuracy, 8.17% one-bug localization accuracy, and 6.15% repair Pass@1. The gap between naming an error and producing a working fix is large enough that those outcomes should not be merged into one success label. The paper also finds performance drops as code context length increases.

Its cases are programmatically injected into repositories, so they may differ from naturally arising AI application faults. The authors note the difficulty of long-range context and the computing demands of repository evaluation. Their manual check samples only part of the corpus, which I would treat as another limitation when interpreting aggregate scores. FaultPilot's first evaluation will be narrower and Python-only, but RepoDebug supports measuring diagnosis and repair as separate stages and preserving repository context. It also motivates a compact evidence window rather than sending an entire codebase to a model.

10. DSDBench [10]

DSDBench concentrates on data science scripts where the line that fails can be downstream of the actual mistake. Yang and colleagues derive tasks from data science benchmarks, inject errors, collect runtime messages, and annotate cause and effect lines. The 1,117 examples include 741 single-error and 376 multi-error cases; the latter average 2.87 errors per example. Their Table 3 shows how sharply performance falls for combined faults: GPT-4o cause-line accuracy moves from 39.0% on single-bug cases to 20.3% on multi-bug cases, while its effect-line accuracy moves from 34.3% to 10.4%. The benchmark scores localization, error type, and message quality separately rather than relying only on repaired code.

The authors acknowledge that their scope is data science scripting and does not yet cover repository-level, multi-file work. The injected faults and use of a model-assisted message rubric also limit how directly scores transfer to real incidents. Nevertheless, this paper exposes a failure mode central to FaultPilot: a traceback may point to a symptom. A secondary stress test can use DSDBench to ask whether the proposed evidence chain identifies the cause line and whether a verification step catches partial fixes when multiple bugs remain.

A.3 Cross-paper comparison

The table separates three kinds of outcomes: correct patches, test-plausible patches, and diagnostic scores. Their denominators and test oracles differ, so the figures are not a single ranking.

Table 1: Cross-paper comparison. The star identifies the base paper.

#	Title	Year / venue	Dataset	Method / agent architecture	Evaluation metrics	Key result	Limitation / research gap
1⋆	RepairAgent [1]	2025 ICSE	Defects4J (835); GitBug-Java (100)	LLM tool use plus state machine	Correct fixes; token cost	164/835 correct; 13/100 correct	Java and pre-existing tests; cost and possible leakage
2	UniDebugger [2]	2025 EMNLP	Defects4J; Codeflaws; QuixBugs	Hierarchical localization and repair agents	Correct versus plausible fixes	197 correct; 286 plausible on Defects4J	Test-driven bugs; orchestration cost
3	COAST [3]	2025 NAACL Findings	DEBUGEVAL (5,712)	Multi-agent training-data synthesis	Task accuracy; repair Pass@1	6.7B model average: 28.7 to 56.4	Teacher-dependent synthetic data
4	SynFix [4]	2025 ACL Findings	SWE-bench Lite and others	Dependency graph and regression tests	Resolved issues; USD; tokens	157/300 Lite; USD 0.56 mean	Weak tests or dependency graph threaten validation
5	MLDebugging [5]	2025 ACL Findings	1,175 Python cases; 126 libraries	Seven-class multi-library benchmark	Test pass rate	Best average: 58.7%	Generated cases; label and setup noise
6	Knowledge injection [6]	2025 ASE	BugsInPy (314)	Staged local, repo, project context	Fixed bugs; Pass@k	250/314 fixed (79%)	Test pass is a proxy for correctness
7	Agent trajectories [7]	2025 ASE	120 runs; 2,822 actions	Agent-log and action-sequence analysis	Steps; tokens; repetition	Mean 6–34 steps by agent	Three unlike agents; correlation only
8	GraphGlue [8]	2025 ASE	1,411 DNN programs	Graph-break mining and self-correction	Optimization; speed; memory	92.63% optimized; 1.23× mean speed	Compiler-specific faults
9	RepoDebug [9]	2025 EMNLP Findings	63 repos; 8 languages	Multi-task repository benchmark	ID and location accuracy; repair Pass@1	Python: 36.77%, 8.17%, 6.15%	Injected errors; long context
10	DSDBench [10]	2025 EMNLP	1,117 data-science cases	Cause/effect-line benchmark	Localization and type accuracy	GPT-4o cause: 39.0% single, 20.3% multi	Script-level injected faults

A.4 Three selected research gaps

Gap 1 — diagnostic evidence in multi-library Python code. MLDebugging's strongest tested model averages 58.7% pass rate [5], while RepoDebug separates much higher bug identification from low Python repair and localization [9]. DSDBench shows that a visible error line can be distinct from the underlying cause [10]. FaultPilot will return an explicit fault category, cause hypothesis, and traceable evidence, and will test whether conditional library/context retrieval improves macro-F1 and cause localization over rules and a one-shot model. A category label by itself is insufficient if it is based on the wrong line or library behavior.

Gap 2 — verified repair with incomplete tests. RepairAgent produces 186 plausible but only 164 correct patches on Defects4J [1]; UniDebugger reports 286 plausible versus 197 correct [2]. SynFix relies on regression suites and warns that poor coverage can make validation unreliable [4]. FaultPilot will run the original reproducer, apply a candidate only in an isolated workspace, rerun target and regression checks, and report which checks passed. A patch will be called *verified by supplied tests*, never universally correct. This gap is measured by target-test repair success, regression rate, and the number of plausible patches rejected by stronger checks.

Gap 3 — bounded decisions and calibrated abstention. Removing RepairAgent's state machine reduced correct fixes from 21 to 14 in its 100-bug ablation [1]. In the trajectory study, failed RepairAgent runs averaged 40 iterations against 22 for successful ones [7]. Staged knowledge injection outperformed dumping all layers at once, 79% versus 65% in one Llama 3.3 setting [6]. FaultPilot will add an explicit state contract, limited retry budget, evidence threshold, and a route to abstain with a diagnostic report. It will measure selective accuracy at a fixed coverage, unsupported repair suggestions, tool calls, latency, and model tokens.

Table 2: Evidence, approach, and measurements for the three selected gaps.

Gap	Supporting evidence	Proposed approach	How it will be measured
1. Grounded diagnosis	MLDebugging [5], RepoDebug [9], DSDBench [10]	Normalize category labels; link traceback, code and library evidence to each diagnosis	Macro-F1 and cause-line accuracy on frozen Python cases; evidence audit
2. Verified repair	RepairAgent [1], UniDebugger [2], SynFix [4]	Isolated candidate patch; target plus regression tests; retain failure traces	Repair-test pass rate, regression rate, plausible-to-verified conversion
3. Bounded abstention	RepairAgent [1], trajectories [7], knowledge injection [6]	State machine, retry and tool budget, stop or abstain when evidence is weak	Selective accuracy at 70% coverage, unsupported suggestions, calls, latency, tokens

B. Project Proposal

B.1 Project, problem and target user

Selected seed and title. Project G2-08 is developed here as *FaultPilot: Evidence-Grounded Debugging for Python AI Applications*. Its scope is Python data science and machine-learning programs that fail while importing packages, moving tensors, transforming data, calling libraries, or running tests. The intended user is a student or developer who can provide a traceback, the relevant code, and a reproducer but cannot immediately tell whether the visible error is the cause or only a symptom.

This is a practical problem because modern AI programs depend on several libraries with different data and device conventions. A type-compatible line may still produce the wrong tensor shape later; a data transformation may fail only when a particular value reaches a model. Existing assistants often return a fluent explanation or a patch without showing which evidence supports it and which tests have run. The literature reinforces that concern: MLDebugging shows weak pass rates in multi-library cases [5], and DSDBench shows a pronounced drop on multiple errors [10]. FaultPilot's proposed end product is a local command-line tool that records the evidence it used, offers a bounded repair attempt, checks it in an isolated environment, and asks for human review when the evidence is insufficient.

The usable final system will accept a case manifest, repository path, traceback, trusted test command, and optional provider configuration. It will emit a machine-readable JSON report and a short human-readable diagnosis with category, suspected cause location, cited code or runtime evidence, proposed edit, test outcomes, and a status of *verified by supplied tests*, *unverified*, or *abstained*. It will not silently modify the user's repository. A VS Code interface is an optional stretch goal; the CLI and its evaluation take priority.

B.2 Base paper, extension and agentic justification

RepairAgent [1] is the base because it provides a complete tool-using repair architecture and ablations that show the value of search and state control. Its result of 164 correct fixes on 835 Java Defects4J cases is relevant evidence for the design, but it is not a target score for a Python benchmark. FaultPilot will adapt three features: an explicit action state, controlled tools, and test feedback. It extends the comparison to multi-library Python cases and asks for a stronger evidence trail and selective abstention. Its diagnostic stage will use a stable taxonomy and retrieved library or project context; its verification stage will record the exact checks passed and failed; its gate will stop when retries repeat the same evidence.

A fixed keyword workflow is appropriate for obvious cases and is the implemented baseline. It cannot choose which file or library evidence to inspect when a traceback is ambiguous, decide whether a failed test calls for another patch, or stop after an unproductive repair loop. A single model call similarly cannot inspect a new test result that does not exist until a tool runs. A plain retrieval pipeline can provide documents but does not decide when to execute a reproducer or whether a candidate patch should be rejected. The proposed agent therefore makes bounded choices among retrieval, diagnosis, patch construction, testing, retry, and abstention. Its behavior is agentic only to the extent that those choices depend on observed results; a long prompt wrapped around the same fixed sequence would not meet the objective.

B.3 Confirmed data and Phase 1 baseline

The primary research source is the public MLDebugging JSONL released with [5], pinned in the repository to commit `0fd4dec` (full commit and SHA-256 checksum are in `faultpilot/data.py`). The repository's ingestion command has downloaded and checked 1,175 records containing code, reference code, tests, runtime feedback, labels, and source task IDs. Its profile found 31 records with multiple recognizable category mentions and two without runtime feedback. The raw benchmark is downloaded from its source rather than copied into this repository. A stable split groups variants by `task_id`: SHA-256 of that ID

modulo five places 234 records in the test partition and 941 in development. That partition is fixed before model comparisons.

The implemented Phase 1 baseline is deliberately small. It reads a local JSON case, matches traceback keywords to shape, device, or dependency categories, and executes a trusted Python reproducer with a five-second timeout. A JSON result records the category, hint, reproduction outcome, and explicit status `diagnosis_only_no_repair_attempted`. Four synthetic cases cover the three known categories and an unknown fallback. All four reproduce their failures and match their labels; four automated tests pass. This is an end-to-end feasibility check, **not** a 100% research accuracy claim. An optional Ollama HTTP adapter and its structured response contract pass a local mock-server test, but no actual model provider has been run. No patch is generated by the baseline.

The initial case schema and evaluation command are documented in the repository README. The larger dataset is an available evaluation source, but its seven category labels do not match the tiny baseline taxonomy one-for-one. The next stage will map a documented subset and preserve ambiguous labels rather than silently treating every record as a single-class example. Only then will a baseline score on the 234 held-out records be reported. This avoids treating data cleaning as a hidden advantage for the proposed agent.

B.4 System architecture and data flow

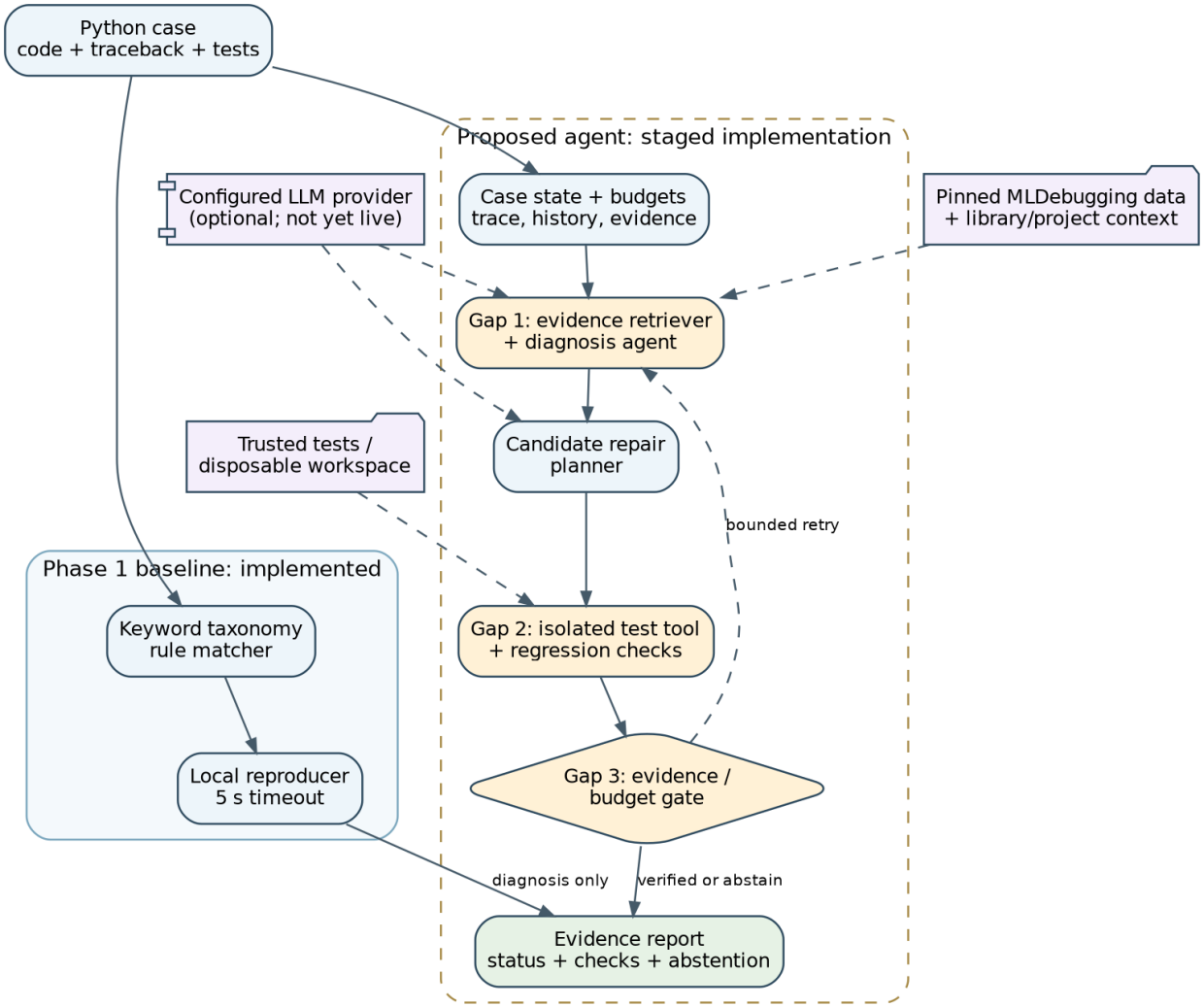


Figure 1. FaultPilot's implemented baseline and proposed agent. Gap 1, Gap 2, and Gap 3 are marked at their intervention points; dashed components are planned.

The proposed *case state* stores the case identifier, repository revision, input traceback, retrieved evidence with source paths, category candidates, current diagnosis, candidate patch, test outcomes, model/tool budgets, attempt count, and final status. Every action produces a structured record, so a later decision can be audited. The tools are a bounded code reader, a taxonomy and library-evidence retriever, a candidate-edit writer limited to a disposable copy, and a trusted test runner. Retrieval will use only the repository and versioned local documentation or benchmark metadata available for the case; it will not treat arbitrary web text as an executable instruction. The optional model provider supplies diagnosis and candidate text, while orchestration and safety decisions remain in code.

Gap 1 is addressed at evidence retrieval and diagnosis. The retriever collects the relevant traceback frame, code span, package/version data where available, and a small number of related symbols or documentation passages. The diagnoser must attach evidence identifiers to its category and cause-line claim. **Gap 2** is addressed when a candidate edit runs in an isolated workspace: the original reproducer must fail before the patch, and the target and regression checks must pass afterward. A missing test or failed setup is reported as unverified. **Gap 3** is addressed by the gate. It permits a limited retry when a new test provides actionable evidence; otherwise it abstains and explains the unresolved question. The implemented baseline follows only the left branch of the diagram—keyword matching, reproducer, report—and cannot claim these three interventions are already complete.

Human controls include explicit approval before applying a candidate to the original checkout, an editable allowlist of test commands, a timeout, and a visible evidence report. Model output will be treated as data; it cannot choose unrestricted shell commands. For Phase 1 the local reproducer runs Python `-c` from trusted fixture JSON, so external or model-generated case files must not be executed by that path. Future benchmark execution will use disposable environments and pinned dependencies.

B.5 Research questions, hypotheses and success criteria

Gap	Narrow empirical question	Testable prediction and success criterion
1	On the held-out MLDebugging task groups, does conditional code/library evidence improve seven-category fault diagnosis over the taxonomy baseline and a one-shot model?	Macro-F1 improves by at least 0.08 absolute over the strongest simpler comparator; at least 80% of a stratified 50-case evidence audit links the diagnosis to a relevant source span.
2	At an equal model-attempt budget, does isolated test feedback improve accepted repair quality over a one-shot candidate?	Target-test success improves by at least 10 percentage points on cases with executable tests, while regressions among accepted patches remain at or below 5%. Report confidence intervals and failures even if targets are missed.
3	Can a bounded evidence gate reduce unsupported repair advice at a fixed answer rate?	At 70% coverage, selective diagnosis accuracy exceeds the always-answer variant by at least 0.10 absolute; median tool calls stay at or below eight per case, with token use and latency reported.

These are prospective targets, not achieved results. Because some source records have missing feedback or more than one label, evaluation will report a full-dataset accounting table and a clearly defined executable subset. The same cases and model budget will be used for paired comparisons.

B.6 Evaluation protocol

The 234 held-out MLDebugging records form the primary evaluation set. The development split may be used to set rules, prompts, and confidence thresholds; the test partition remains frozen. Category scoring will use macro-F1 and per-class precision/recall, with multi-label cases evaluated using a declared multi-label protocol and separately reported rather than silently dropped. Diagnosis evidence will be audited on 50 stratified test cases against the provided runtime feedback, code, and relevant source path. Repair evaluation will use the benchmark's supplied tests where they can be run in a pinned environment. A patch that passes the target test but fails a separate regression test is counted as a regression, not a success. Cases whose dependencies cannot be reproduced will be reported as setup failures and excluded only from repair-rate denominators, with the exclusion count shown.

Comparators are: (B0) the current deterministic taxonomy baseline after a documented seven-class mapping; (B1) one model call using the traceback and local code; (B2) the same model with conditional retrieval; (B3) a bounded RepairAgent-style read/edit/test loop; and (B4) B3 with the evidence gate and abstention. The same provider, temperature, maximum attempts, and token budget will be used wherever a model is involved. Ablations remove retrieval, test feedback, or abstention one at a time. A secondary DSDBench analysis will focus on cause versus effect localization for multi-bug data science cases if its published runtime assets reproduce locally; the primary MLDebugging test remains the committed evaluation choice.

Results will report macro-F1, cause-line accuracy where labels exist, target-test pass, regression rate, answer coverage, selective accuracy, median and 95th-percentile latency, tool calls, and model tokens/cost. Uncertainty will be estimated with bootstrap intervals grouped by source task, so related variants are resampled together. A paired significance test or interval will compare each ablation on exactly the same cases. Error analysis will separate wrong categories, missing evidence, failed environment setup, patch overfitting, and budget exhaustion. The four synthetic Phase 1 cases will remain a smoke check and will never enter the research test set.

B.7 Milestones, resources and feasibility

Stage	Deliverable	Main risk and fallback
Phase 1, completed foundation	Local case pipeline, deterministic baseline, tests, optional model-provider adapter with a mock contract test, pinned MLDebugging ingestion and profile, architecture and review	The present result proves plumbing only; it does not establish model or repair quality.
Next 1–2 weeks	Normalize benchmark labels; create executable subset and reproducible environment; freeze test groups	Dependency setup may fail for some examples. Publish an accounting table and use the reproducible subset without changing the held-out rule.

Stage	Deliverable	Main risk and fallback
Following 2 weeks	Run the configured model, implement one-shot and retrieval comparators, and attach evidence links	API cost or access may constrain experiments. Use a smaller development sample, then one frozen test run at a fixed budget.
Following 2 weeks	Add isolated patch verification, bounded retries, and abstention	Unsafe or flaky tests can give false assurance. Use disposable execution, timeouts, repeated checks, and an `unverified` status.
Final evaluation and paper	Ablations, confidence intervals, failure analysis, release of scripts and manifests	If full repair remains too costly, report a rigorous diagnosis/abstention study and treat repair as a measured limitation rather than imply success.

This is a solo project. Fazal Zaman Khan is responsible for literature review, data and license checks, implementation, test environments, evaluation, writing, and the final submission. The work should run on a laptop for rules, data profiling, and a small development set; model-driven experiments may require an API or hosted GPU and therefore need strict call and token limits. No secret belongs in Git. Current machine constraints have prevented a live provider run, so that remains an explicit next-stage validation task rather than an invented Phase 1 result.

B.8 Potential publication direction

If the final controlled comparison supports the hypotheses, the contribution could be an evidence-grounded, budgeted repair protocol for multi-library Python AI faults, together with a reproducible split, category-cleaning audit, and failure taxonomy. The contribution would be stronger if independent cases beyond the source benchmark confirm the result and the code, prompts, and test manifests can be released. A negative result—retrieval or verification adding cost without improving accepted repairs—would also be worth reporting if the evaluation is transparent. The current four-case smoke run is too small for any publication claim.

References

[1] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," in *Proc. 2025 IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, 2025, pp. 2188–2200, doi: 10.1109/ICSE55347.2025.00157.

[2] C. Lee *et al.*, "UniDebugger: Hierarchical Multi-Agent Framework for Unified Software Debugging," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 18237–18266, doi: 10.18653/v1/2025.emnlp-main.921.

[3] W. Yang *et al.*, "COAST: Enhancing the Code Debugging Ability of LLMs through Communicative Agent Based Data Synthesis," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 2570–2585, doi: 10.18653/v1/2025.findings-naacl.139.

[4] X. Tang *et al.*, "SynFix: Dependency-Aware Program Repair via RelationGraph Analysis," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 4878–4894, doi: 10.18653/v1/2025.findings-acl.252.

[5] J. Huang *et al.*, "MLDebugging: Towards Benchmarking Code Debugging Across Multi-Library Scenarios," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 5866–5879, doi: 10.18653/v1/2025.findings-acl.305.

[6] R. Ehsani, E. Parra, S. Haiduc, and P. Chatterjee, "Hierarchical Knowledge Injection for Improving LLM-based Program Repair," in *Proc. 2025 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2025, pp. 1440–1452, doi: 10.1109/ASE63991.2025.00122.

[7] I. Bouzenia and M. Pradel, "Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories," in *Proc. 2025 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2025, pp. 2846–2857, doi: 10.1109/ASE63991.2025.00234.

[8] H. Wang *et al.*, "Fixing Broken Graphs: LLM-Powered Automatic Code Optimization for DNN Programs," in *Proc. 2025 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2025, pp. 1718–1730, doi: 10.1109/ASE63991.2025.00144.

[9] J. Liu *et al.*, "RepoDebug: Repository-Level Multi-Task and Multi-Language Debugging Evaluation of Large Language Models," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 23784–23813, doi: 10.18653/v1/2025.findings-emnlp.1294.

[10] Z. Yang, S. Wang, Y. Yan, and Y. Deng, "Why Stop at One Error? Benchmarking LLMs as Data Science Code Debuggers for Multi-Hop and Multi-Bug Errors," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 17366–17381, doi: 10.18653/v1/2025.emnlp-main.878.

[11] ICSE 2025, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," Research Track program record. [Online]. Available: https://conf.researchr.org/details/icse-2025/icse-2025-research-track/160/RepairAgent-An-Autonomous-LLM-Based-Agent-for-Program-Repair. Accessed: Oct. 9, 2026.

[12] Computing Research and Education Association of Australasia, "CORE 2023 Conference Rankings," entry for International Conference on Software Engineering (ICSE), rank A*. [Online]. Available: https://portal.core.edu.au/conf-ranks/?by=all&page=1&search=&sort=arank&source=CORE2023. Accessed: Oct. 9, 2026.